

EasyQ Automation Project Guide

Selenium Java TestNG framework for EasyQ beta environment

Field	Value
Project folder	C:\Users\SAURABH\Documents\Codex\2026-06-12\i-want-to-work-on-easyq\outputs\easyq-automation
Beta URL	https://beta.easyqsolutions.com/#!/easyqsolutions/login
Repository	https://github.com/SAURABH188/easyq-automation..git
Branch	master
Prepared for	QA lead self-use and demo walkthrough
Document date	July 1, 2026

Security note: This guide intentionally does not include real account passwords. Store passwords in Eclipse environment variables or secrets.local.properties, and never commit them to GitHub.

Purpose: This guide explains how the EasyQ automation project is structured, how to set it up, how scripts are written, how to run module suites, and how to present the framework during a demo.

Table of Contents

- Project Snapshot
- Framework and Tool Stack
- Project Folder Structure
- Configuration and Environment Variables
- Step-by-Step Setup in Eclipse
- How Scripts Are Written
- Wait Strategy and Stability Handling
- TestNG Suites and Module Coverage
- Running Tests
- Dynamic Workflow and Role-Based Testing
- Reporting and Debugging
- GitHub Workflow
- Demo Talking Points
- Maintenance Checklist
- Appendix A. Useful Commands
- Appendix B. Current Suite Map

1. Project Snapshot

EasyQ automation is a Selenium WebDriver framework built in Java for validating the EasyQ beta web application. It uses TestNG for test organization, Maven for dependency management, WebDriverManager for browser driver setup, and Eclipse as the primary IDE.

Area	Current Details
Application under test	EasyQ beta web app
Environment	https://beta.easyqsolutions.com/#/easyqsolutions/login
Primary browser	chrome
Automation style	Selenium WebDriver + TestNG module suites
Current test classes	21
Current @Test methods detected	783
Failure artifacts	Console failure reason and screenshots under test-output/screenshots

Automation Goals

- Validate login, dashboard and major QMS modules in beta.
- Map automation methods back to manual test case IDs where available.
- Use role-based users for Admin, Document Controller and Assignee behavior.
- Handle dynamic screens by checking real UI states instead of blindly skipping workflows.
- Generate useful failure details for fast debugging during demo and local execution.

2. Framework and Tool Stack

Tool	Purpose	Current Version / Source
Java	Programming language used for Selenium scripts.	17
Maven	Dependency management, compile lifecycle and suite execution.	pom.xml
Selenium WebDriver	Browser automation and UI interaction.	4.33.0
TestNG	Test annotations, priorities, suites, listeners and reports.	7.8.0
WebDriverManager	Automatically resolves browser drivers.	6.1.0
SLF4J Simple	Logging provider to avoid missing logger warnings.	1.7.36
Eclipse IDE	Primary authoring and run environment.	Run as TestNG Suite
GitHub	Source control and sharing.	origin remote configured

The Maven project keeps framework versions in pom.xml properties. The surefire plugin can run any suite file by changing the suiteXmlFile property.

3. Project Folder Structure

Path / Pattern	Purpose
src/test/java/base	Base test setup, browser creation and shared start/stop flow.
src/test/java/pages	Page Object classes, currently including LoginPage.
src/test/java/tests	TestNG test classes for login, dashboard and QMS modules.

Path / Pattern	Purpose
src/test/java/utils	Reusable helpers for waits, actions, dynamic workflows, config and screenshots.
src/test/resources	Framework config and optional local secrets file support.
testng*.xml	Suite files for full run and module-wise execution.
test-output/screenshots	Failure screenshots generated by TestListener.
easyq-automation/ pom.xml testng.xml testng-*.xml src/test/java/base/BaseTest.java src/test/java/pages/LoginPage.java src/test/java/tests/*.java src/test/java/utils/*.java src/test/resources/config.properties test-output/screenshots/	

4. Configuration and Environment Variables

config.properties

Key	Current Value / Use
baseUrl	https://beta.easyqsolutions.com/##easyqsolutions/login
browser	chrome
explicitWait	45
actionDelayMs	1500
pageSettleMs	1200
highlightActions	true
allowWorkflowMutations	false
allowPasswordResetMutation	false

Passwords should be entered in Eclipse Run Configuration > Environment, as system properties, or in a local secrets.local.properties file that is not committed.

Variable	Use
EASYQ_ADMIN_PASSWORD or EASYQ_PASSWORD	Admin password used for valid login scenarios.
EASYQ_DOC_CONTROLLER_PASSWORD	Document Controller role password.
EASYQ_ASSIGNEE_SWATI_PASSWORD	Assignee role password for role-restriction checks.
EASYQ_TEST_USER_*	Disposable automation user data for User Management workflows.
EASYQ_ALLOW_WORKFLOW_MUTATIONS	Allows create/edit/delete workflow tests when true.
EASYQ_VISUAL_DELAY_MS	Override for visible action delay while watching automation run.
EASYQ_PAGE_SETTLE_MS	Override for post-load DOM settle delay.

Recommended demo setting: Use EASYQ_VISUAL_DELAY_MS=1500 or 2000 when showing automation to the team, so actions are visible.

5. Step-by-Step Setup in Eclipse

1. Install or select JDK 17 in Eclipse. Confirm Project > Properties > Java Compiler uses Java 17.
2. Open Eclipse and choose File > Import > Maven > Existing Maven Projects.
3. Select the easyq-automation folder under the Codex output directory.
4. Let Maven download dependencies. If the Maven cache is locked, close Eclipse/Java processes and retry Maven Update.
5. Install or enable the TestNG plugin in Eclipse if TestNG run options are not visible.
6. Open Run Configurations > TestNG > your suite configuration.
7. Set Environment variables such as EASYQ_ADMIN_PASSWORD, role passwords and optional visual delay.
8. Run testng-user-management.xml or any module XML as a TestNG Suite.
9. Review Eclipse Console output and screenshots in test-output/screenshots when failures occur.

Eclipse Environment Example

```
EASYQ_ADMIN_PASSWORD=<admin password>
EASYQ_DOC_CONTROLLER_PASSWORD=<doc controller password>
EASYQ_ASSIGNEE_SWATI_PASSWORD=<assignee password>
EASYQ_VISUAL_DELAY_MS=1500
EASYQ_PAGE_SETTLE_MS=1200
```

6. How Scripts Are Written

Each script is a TestNG Java class. Test methods are marked with `@Test`, usually with priority and description. Manual test case IDs are placed near the script as normal comments where available.

Typical Test Flow

1. Start Chrome using WebDriverManager.
2. Open the EasyQ beta login page.
3. Wait for page load and visible fields.
4. Login using configured username and password.
5. Navigate to the target module by sidebar, profile dropdown or route fallback.
6. Wait for the module to render and loaders to finish.
7. Perform the action or assertion for the manual test scenario.
8. On failure, print the failure reason and capture a screenshot.

Script Pattern Example

```
@Test(priority = 1, description = "Verify User Management page loads successfully")
// Manual Test Case ID: TC279
public void verifyUserManagementPageLoadsSuccessfully() {
    Assert.assertTrue(
        wait.until(ExpectedConditions.visibilityOfElementLocated(userManagementTitle)).isDisplayed(),
        "User Management page should load successfully"
    );
}
```

Locator Strategy

- Use flexible XPath/CSS selectors when stable IDs are not available in beta UI.
- Prefer visible text, aria-label, title, role and formcontrolname over brittle absolute XPath.

- Future improvement: ask developers to add stable data-testid attributes for important controls.

7. Wait Strategy and Stability Handling

The beta environment can load slowly. The framework combines explicit waits, app-load waits, visible loader checks, DOM settle delay and action observation delay.

Layer	Behavior
BaseTest	Sets browser, page-load timeout, script timeout, opens base URL and waits for app load.
WaitHelper	Waits for document.readyState, Angular stability, loader disappearance and DOM settle.
ActionHelper	Waits before click/type, scrolls/highlights elements and pauses after each action.
TestNG XML	configfailurepolicy=continue helps avoid a skip cascade after setup/browser issues.
User Management special handling	If Chrome is manually closed or setup fails, the test is failed clearly instead of silently skipped.

Why this matters: If scripts run faster than the platform can render, Selenium looks for elements too early and the test breaks.

8. TestNG Suites and Module Coverage

The project currently contains 21 Java test classes and 783 detected @Test methods. Module suite XML files are available so you can run one area at a time.

Module	Class	Suite XML	Tests	Focus
Active Logs	EasyQActiveLogsTest.java	testng-active-logs.xml	34	Logs, grouping, date/time, scrolling and pagination.
CAPA	EasyQCapaTest.java	testng-capaxml	42	Access, draft/review/approval, lifecycle and role restrictions.
Complaint Management	EasyQComplaintManagementTest.java	testng-complaint-management.xml	40	Complaint form, save/submit, upload, CAPA linkage, filters and actions.
Credential Validation	EasyQCredentialValidationTest.java	testng-credential-validation.xml	1	Configured role users can authenticate.
Dashboard Details	EasyQDashboardDetailsTest.java	testng-dashboard.xml	104	Widget counts, navigation, severity, role-based and null/large data.
Dashboard	EasyQDashboardTest.java	testng-dashboard.xml	25	Dashboard load, reload, widgets, toggles and basic layout.
Document Management	EasyQDocumentManagementTest.java	testng-document-management.xml	44	Upload, drafts, review/approval, download, versions and obsolete flow.
Dynamic Workflow	EasyQDynamicWorkflowTest.java	testng-dynamic-workflow.xml	14	State-aware workflows for dynamic UI conditions.
Homepage / Dashboard Smoke	EasyQHomepage.java	testng.xml	7	Dashboard card visibility smoke checks.
Login + Forgot Password Remaining	EasyQLoginPasswordRemainingTest.java	testng-login-password-remaining.xml, testng.xml	21	Additional login, validation, browser and forgot-password scenarios.
Login UI/UX Comprehensive	EasyQLoginUiUxComprehensiveTest.java	testng-login-ui-ux.xml	40	Login UI, validation, keyboard, responsive and edge cases.
Login Validation	EasyQLoginValidation.java	testng.xml	19	Manual login IDs, valid/invalid login and forgot-password navigation.
Management Review	EasyQManagementReviewTest.java	testng-management-review.xml	48	MR assignment, schedule, MOM, approval and task flow.
Notifications	EasyQNotificationsTest.java	testng-notifications.xml	77	Panel, read/unread, ordering, show more, redirection and data.

Module	Class	Suite XML	Tests	Focus
Products	EasyQProductsTest.java	testng-products.xml	19	List, search, filters, status, delete, duplicates and text handling.
Quality Objective	EasyQQualityObjectiveTest.java	testng-quality-objective.xml	40	Initiation, draft, review, approval, versions and access checks.
Quality Policy	EasyQQualityPolicyTest.java	testng-quality-policy.xml	39	Policy draft/review/approval, active/inactive and versioning.
Responsibility & Authority	EasyQResponsibilityAuthorityTest.java	testng-responsibility-authority.xml	39	Rows, drafts, review/approval, downloads and access.
Training	EasyQTrainingTest.java	testng-training.xml	68	Assignments, document linkage, completion, counts, roles and dynamic states.
User Management	EasyQUserManagementTest.java	testng-user-management.xml	60	Users, licenses, tabs, add/edit/disable, roles, access and cleanup.
Login Smoke	LoginTest.java	testng.xml	2	Basic login page and valid login smoke checks.

9. Running Tests

Run from Eclipse

1. Right-click the suite file, for example testng-user-management.xml.
2. Choose Run As > TestNG Suite.
3. Watch the browser actions and Eclipse Console logs.
4. If one test fails, open the failure reason and screenshot path printed in the console.
5. For failed-only reruns, use Eclipse TestNG failed suite output when available, or run the specific method/class.

Run from Maven

```
mvn test
mvn test -DsuiteXmlFile=testng-user-management.xml
mvn test -DsuiteXmlFile=testng-login-ui-ux.xml
```

Maven cache issue: If Maven reports AccessDeniedException under .m2, close Eclipse/Java processes and remove the affected plugin folder only after verifying the path.

10. Dynamic Workflow and Role-Based Testing

Some EasyQ workflows are dynamic: a module may show existing data, no data, restricted access, draft state, review state or approved state. The framework checks valid states instead of directly skipping workflows.

- DynamicWorkflowHelper checks for meaningful page states such as Draft, Review, Approved, Pending, Completed, Closed, Open, No Data or Access Denied.
- Mutation-heavy tests are controlled by allowWorkflowMutations or EASYQ_ALLOW_WORKFLOW_MUTATIONS.
- Role-based tests use configured Admin, Document Controller and Assignee accounts.
- Disposable user tests use EASYQ_TEST_USER_* variables and include cleanup behavior.

Role Type	Username Key	Password Key
Admin	EASYQ_ADMIN_USERNAME	EASYQ_ADMIN_PASSWORD or EASYQ_PASSWORD
Document Controller	EASYQ_DOC_CONTROLLER_USERNAME	EASYQ_DOC_CONTROLLER_PASSWORD

Role Type	Username Key	Password Key
Assignee	EASYQ_ASSIGNEE_SWATI_USERNAME	EASYQ_ASSIGNEE_SWATI_PASSWORD
Other Assignees	EASYQ_ASSIGNEE_*_USERNAME	matching password variable if used

11. Reporting and Debugging

TestListener prints failed test name, test data, failure reason and screenshot path. Several tests also print navigation or login step logs to make stuck points visible.

```

FAILED TEST: verifyUserManagementPageLoadsSuccessfully
FAILURE REASON: <root cause>
SCREENSHOT: C:\...\test-output\screenshots\verifyUserManagementPageLoadsSuccessfully.png

```

Common Issues and Fixes

Issue	Meaning	Fix
EASYQ_PASSWORD is required	Password variable is missing.	Set EASYQ_ADMIN_PASSWORD or EASYQ_PASSWORD in Eclipse.
Login stays on login page	Password may be wrong, account inactive, or field/click issue.	Verify credentials manually, then rerun.
CDP implementation warning	Chrome is newer than Selenium devtools package.	Usually non-blocking unless DevTools APIs are used.
NoSuchWindowException	Chrome was manually closed or crashed.	Rerun; teardown is now safer and next test starts fresh.
Timeout waiting for element	Page, module or control did not render in time.	Use module suite, confirm navigation, increase visual delay/wait.
Unsupported class file major version	Java/Maven/TestNG runtime mismatch.	Use JDK 17 consistently in Eclipse and Maven.
.m2 AccessDeniedException	Maven cache folder locked or permission issue.	Close Java/Eclipse processes; remove only affected plugin cache if it exists.

12. GitHub Workflow

The project is connected to GitHub with origin remote configured. Use small commits after stable changes, and avoid committing secrets.local.properties, screenshots containing sensitive data, or real passwords.

Item	Value
Remote	https://github.com/SAURABH188/easyq-automation..git
Current branch	master
Recommended commit scope	One framework/module change at a time

Manual Push Steps from Eclipse

1. Right-click project > Team > Commit.
2. Review changed files and enter a clear commit message.
3. Click Commit and Push, or Commit first and push later.
4. If Push Ref Specification appears empty, choose Add All Branches Spec or select master -> master.
5. Confirm Push Results shows master -> master or your current branch pushed successfully.

Command Line Backup


```
git status
git add .
git commit -m "Update EasyQ automation framework waits and documentation"
git push origin master
```

13. Demo Talking Points

1. Start by explaining the purpose: automate EasyQ beta validation across login, dashboard and QMS modules.
2. Show pom.xml and explain Java, Selenium, TestNG and WebDriverManager.
3. Show config.properties and explain that passwords are outside source code.
4. Show testng XML files and explain module-wise execution.
5. Open one test class and point out @Test description and manual test case ID comments.
6. Show BaseTest, ActionHelper and WaitHelper to explain stability handling.
7. Run a small suite such as login UI/UX or User Management page load.
8. Show console output and screenshot behavior if a test fails.
9. Close with next improvements: stable locators, CI, reports and cleaner page objects.

14. Maintenance Checklist

Checklist Item	Why It Matters
Keep credentials outside GitHub.	Prevents password leakage.
Use module XML while debugging.	Faster feedback than running the full suite.
Prefer stable locators.	Reduces failures caused by UI layout changes.
Add manual test case ID comments.	Keeps automation linked with QA test cases.
Capture useful assertion messages.	Makes failed tests understandable without reopening code.
Use dynamic state handling for workflows.	Avoids false failures when application data state changes.
Review screenshots after failures.	Fastest way to confirm script, data, role or app-loading issue.
Commit after a passing compile.	Keeps GitHub history clean and demo-safe.

Appendix A. Useful Commands

```
# Run full suite if Maven cache is healthy
mvn test

# Run a module suite
mvn test -DsuiteXmlFile=testng-user-management.xml

# Check Git status
git status

# Push current branch
git push origin master

# Useful Eclipse run environment examples
EASYQ_ADMIN_PASSWORD=<password>
EASYQ_VISUAL_DELAY_MS=1500
EASYQ_PAGE_SETTLE_MS=1200
```

Appendix B. Current Suite Map

Suite File	Suite Name	Classes
testng-active-logs.xml	easyQ Active Logs Tests	EasyQActiveLogsTest
testng-capa.xml	easyQ CAPA Tests	EasyQCapaTest
testng-complaint-management.xml	easyQ Complaint Management Tests	EasyQComplaintManagementTest
testng-credential-validation.xml	easyQ Credential Validation	EasyQCredentialValidationTest
testng-dashboard.xml	easyQ Dashboard Tests	EasyQDashboardTest, EasyQDashboardDetailsTest
testng-document-management.xml	easyQ Document Management Tests	EasyQDocumentManagementTest
testng-dynamic-workflow.xml	easyQ Dynamic Workflow Automation	EasyQDynamicWorkflowTest
testng-login-password-remaining.xml	easyQ Login Password Remaining Tests	EasyQLoginPasswordRemainingTest
testng-login-ui-ux.xml	easyQ Login UI UX Automation	EasyQLoginUiUxComprehensiveTest
testng-management-review.xml	easyQ Management Review Tests	EasyQManagementReviewTest
testng-notifications.xml	easyQ Notifications Tests	EasyQNotificationsTest
testng-products.xml	easyQ Products Tests	EasyQProductsTest
testng-quality-objective.xml	easyQ Quality Objective Tests	EasyQQualityObjectiveTest
testng-quality-policy.xml	easyQ Quality Policy Tests	EasyQQualityPolicyTest
testng-responsibility-authority.xml	easyQ Responsibility Authority Tests	EasyQResponsibilityAuthorityTest
testng-training.xml	easyQ Training Tests	EasyQTrainingTest
testng-user-management.xml	easyQ User Management Tests	EasyQUserManagementTest
testng.xml	easyQ Beta Automation	LoginTest, EasyQHomepage, EasyQLoginValidation, EasyQLoginPasswordRemainingTest

Appendix C. Recommended Next Improvements

- Ask developers to add data-testid attributes to key buttons, inputs, menus and module links.
- Move remaining module classes toward a common BaseTest and page-object structure.
- Add ExtentReports or Allure for richer demo reporting.
- Create CI execution for nightly smoke suites once credentials are stored in CI secrets.
- Keep one stable smoke suite separate from long workflow mutation tests.